1 a. Define Distributed Systems. List and explain the significant consequences of Distributed Systems

A Distributed System is a collection of independent computers that appear to users as a single coherent system. These computers communicate and coordinate their actions by passing messages to achieve a common goal.

Significant Consequences of Distributed Systems

Concurrency

Distributed systems allow multiple components to run concurrently, enabling parallel execution of tasks.

Consequence: Increased performance and scalability but also challenges in synchronization and resource sharing.

Lack of a Global Clock

Distributed systems do not have a single, universal clock to synchronize all nodes.

Consequence: Events must be ordered logically using techniques like Lamport timestamps or vector clocks, making time coordination complex.

Independent Failures

Each component in a distributed system can fail independently.

Consequence: Fault tolerance mechanisms such as replication, recovery, and consensus algorithms are essential to ensure system reliability and availability.

Resource Sharing

Distributed systems allow sharing of hardware, software, and data resources across different locations.

Consequence: Efficient management of shared resources is critical to prevent conflicts and deadlocks.

Scalability

The system can be scaled by adding more nodes without degrading performance.

Consequence: Design decisions must account for performance bottlenecks and network latency.

Heterogeneity

Distributed systems consist of different hardware, operating systems, and networks.

Consequence: Middleware solutions are needed to handle interoperability and ensure a seamless user experience.

Transparency

The system should hide the complexities of distribution from the user (e.g., access, location, replication, failure transparency).

Consequence: Achieving complete transparency is challenging and often involves trade-offs in performance and reliability.

b. Discuss the key challenges in the Distributed Systems.

Heterogeneity

- **Different hardware, software, and networks are used.**
- **Challenge: Making them work together.**
- **Solution: Use tools like middleware to hide differences.**

Scalability

- **The system needs to handle more users or devices without slowing down.**
- **Challenge: Avoid performance issues as it grows.**
- **Solution: Use techniques like load balancing and caching.**

Fault Tolerance

- **Components can fail at any time.**
- **Challenge: Keeping the system running even when something breaks.**
- **Solution: Use backups, replication, and recovery mechanisms.**

Concurrency

- **Many tasks happen at the same time.**
- **Challenge: Prevent conflicts and ensure proper coordination.**
- **Solution: Use locks and synchronization methods.**

Security

- **The system is open to attacks.**
- **Challenge: Protecting data and preventing unauthorized access.**
- **Solution: Use encryption, authentication, and secure protocols.**

Latency

- **Communication between nodes takes time.**
- **Challenge: Reducing delays.**
- **Solution: Use faster networks and local data storage.**

Consistency

- **Data should be the same everywhere.**
- **Challenge: Ensuring all users see updated data.**
- **Solution: Use consistency models like eventual or strong consistency.**

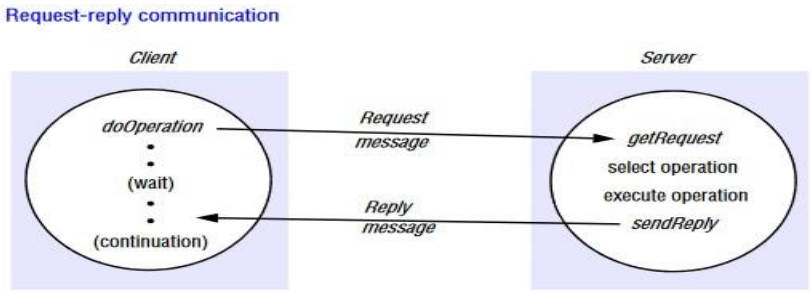
Dynamic Changes

- **Nodes can join or leave anytime.**
- **Challenge: Adapting to changes without breaking the system.**
- **Solution: Use flexible protocols.**

c. Explain the Reply-Request Protocol used in the Distributed Systems for process communication.

Request-Reply Protocols

In remote invocation, a **request-reply protocol** governs the communication between the client and the server. The client sends a request to the server, askingit to perform a task, and the server replies with the result once the task is complete.



Steps in the Request-Reply Protocol:

Client Side:

1. **doOperation (Initiating the Request):**
   - The client starts by performing an operation or task that involves communicating with the server.
   - To perform the operation, the client formulates a **request message**. This message includes all the necessary details for the server to understand what action to take. For example, it may include the operation type, parameters, or any other data needed to fulfill the request.
2. **Send Request Message:**
   - The client sends the **request message** to the server over a communication network (e.g., TCP, HTTP, etc.).
   - This message is delivered to the server using some form of transport protocol.
3. **Wait (Blocking Until Response):**
   - After the client sends the request, it enters a waiting or **blocking** state. The client essentially "pauses" its own operations until it receives a response from the server.
   - This is a **synchronous** form of communication, meaning the client cannot continue its work until the server has processed the request and replied.
4. **Receive Reply Message:**
   - The client eventually receives a **reply message** from the server. This message contains the result of the operation or any relevant data.
   - Once the reply is received, the client exits the waiting state.
5. **Continuation:**
   - After receiving the server's response, the client processes the result and continues with its own workflow or logic.
   - This is where the operation concludes, and the client is free to make further requests or continue with other tasks.

Server Side:

1. **getRequest (Receiving the Request):**
   - The server listens for incoming **request messages** from clients. Once a request is received, the server begins processing it.
   - The server reads the request message, which contains details about the operation the client wants it to perform.
2. **select operation (Determining the Task):**

- The server decodes the request message to determine what action needs to be taken.
- Based on the request's content, the server selects the appropriate operation or service to fulfill the client's needs. For instance, the request might ask the server to retrieve data from a database, perform a calculation, or carry out some other specific operation.

3. **execute operation (Performing the Task):**
    - Once the operation is selected, the server executes the requested task. This could involve:
        1. Fetching data from a database.
        2. Running a program or method.
        3. Performing complex computations.
        4. Interacting with other services.
4. **sendReply (Responding to the Client):**
    - After the server completes the requested operation, it packages the results (or status of the operation) into a **reply message**.
    - The **reply message** is sent back to the client via the same communication channel used for the request.
    - The server completes the cycle by fulfilling the client's request, after which it is ready to handle other client requests.

Q2 a. Discuss the design issues for Remote Procedure Call (RPC)

Remote Procedure Call (RPC) allows a program to invoke a procedure on a remote system as if it were a local call. However, designing RPC involves addressing several challenges to ensure efficiency, transparency, and reliability. The key design issues are:

Design Issues for RPC (Simplified)

1. **Transparency**
    - **Remote calls should feel like local ones.**

- **Solution: Use stubs to hide communication details.**

2. **Data Conversion**
    - **Convert data for transmission (marshaling/unmarshaling).**
    - **Solution: Use standard formats like JSON or Protocol Buffers.**
3. **Communication Failures**
    - **Handle network or remote server failures.**
    - **Solution: Use retries, timeouts, and error detection.**
4. **Latency**
    - **Remote calls are slower than local ones.**
    - **Solution: Use caching or asynchronous calls.**
5. **Binding**
    - **Connect client and server.**
    - **Solution: Use dynamic binding or service directories.**
6. **Security**
    - **Protect data and prevent unauthorized access.**
    - **Solution: Use encryption and authentication.**
7. **Concurrency**
    - **Handle multiple requests at once.**
    - **Solution: Use threads and synchronization.**
8. **Scalability**
    - **Handle more users without slowing down.**
    - **Solution: Load balancing and efficient resource use.**
9. **Error Handling**
    - **Manage errors during remote calls.**
    - **Solution: Use clear error codes or exceptions.**

b. Illustrate the implementation of Remote Procedure Call (RPC) in a Distributed Systems environment.

c Discuss the implantation of Remote method invocation (RMI).

----Refer notes-----

3 a. Review the characteristics of file systems

1. **Hierarchical Structure**
    - **Files are organized in a tree-like directory structure.**
    - **Benefit: Makes file navigation and management easier.**
2. **Data Storage**
    - **Files store data in blocks on storage devices.**
    - **Characteristic: File systems optimize storage space by managing block allocation.**
3. **File Naming**
    - **Files have names to make them identifiable.**
    - **Characteristic: Supports extensions and naming rules based on the operating system.**
4. **Access Methods**
        - **Sequential Access: Data is read/written in order.**
        - **Random Access: Data can be accessed directly.**
5. **Security and Permissions**
    - **Controls access to files using user/group permissions (e.g., read, write, execute).**
    - **Benefit: Protects files from unauthorized access.**
6. **Fault Tolerance**
    - **Handles errors and recovers data during failures.**
    - **Characteristic: May use journaling or backups for reliability.**
7. **Scalability**
    - **Handles increasing storage capacity and number of files efficiently.**
    - **Characteristic: Supports large files and multiple users.**
8. **Portability**
    - **Allows file systems to be used across different platforms.**
    - **Characteristic: Formats like FAT, NTFS, and ext4 vary in compatibility.**
9. **Concurrency**

- **Supports multiple users or processes accessing files simultaneously.**
- **Benefit: Manages conflicts and ensures data integrity.**

11. **Caching**

- **Temporarily stores frequently accessed data for faster retrieval.**
- **Benefit: Improves system performance.**

12. **File Sharing**

- **Allows multiple users to share files over a network.**
- **Characteristic: Uses locking mechanisms to avoid conflicts.**

b. Discuss the key requirements for Distributed File System.

1. Transparency

Transparency in DFS aims to provide a seamless experience to users as if they were dealing with a local file system. It involves several types:

Location Transparency: Users should not need to know the physical location of files. Regardless of where a file is stored (locally, on a remote server, or even replicated across servers), the file should appear to reside in a single, consistent namespace. The system automatically locates and accesses the file without user intervention

Access Transparency: The method of accessing files (opening, reading, writing, etc.) should be identical for both local and remote files. DFS achieves this by making remote file operations look just like local ones through a uniform interface.

Failure Transparency: A robust DFS can mask the effects of failures, such as server or network outages. When part of the system goes down, the DFS can reroute access requests to replicated data on other servers or provide backup mechanisms to ensure continuity of service.

Replication Transparency: If a file is replicated on multiple servers, the user should not need to know this. Replicas ensure that even if one server fails, the file is still available from another server. Users and applications should interact with the file as though it were a single instance, not multiple copies

Migration Transparency: Files can be moved between servers or storage systems without the user being aware of the move. The system automatically keeps track of the new location and provides access

2. Scalability

Distributed systems often need to handle large numbers of clients and growing data volumes. A well-designed DFS is scalable in terms of:

Storage capacity: As more data needs to be stored, new servers can be added to the system, allowing more storage space to be made available without significant reconfiguration.

3. Replication

Replication ensures data is copied across multiple nodes or servers to improve both performance and fault tolerance. This feature includes several key aspects:

High Availability: In case one server or node goes down, other replicas are available to serve the data without interruption.

Improved Performance: Replication allows users to access data from the closest replica, reducing latency and improving speed for geographically dispersed users.

4. Consistency

Maintaining consistency across multiple replicas of the same file is a key challenge.

All users see the same version of a file at all times. Any changes made to a file are immediately reflected across all replicas. While this ensures data integrity, it can lead to performance bottlenecks, especially in large systems with many users or geographically distant nodes.

5. Concurrency Control

DFS must allow multiple users to access and modify files concurrently. The key challenge here is to ensure that the system maintains data integrity and correctness without conflicts when multiple processes access the same file:

6. Fault Tolerance and Recovery

A DFS must be resilient to failures in the network, storage devices, or servers. This feature ensures continuous operation even when components fail:

Replication: As mentioned, file replication across multiple servers ensures that if one server fails, the file is still accessible from another.

Failover Mechanisms: DFS can automatically switch to backup servers or replicas in case of failure. Advanced systems may use techniques such as checkpointing, where the system periodically saves the state of ongoing operations to help recover in case of failure.

Data Recovery: In case of hardware failure or data corruption, the system must be able to recover lost files or roll back to previous versions. Backup and replication play a key role here.

7. Security

Security is a critical concern in DFS, particularly since data is transmitted over potentially insecure networks:

Access Control: DFS should implement robust authentication and authorization mechanisms, ensuring that only authorized users can access or modify files. This can involve role-based access controls, identity management, and user authentication protocols.

Encryption: Data transmitted across the network and stored on servers should be encrypted to prevent unauthorized access or tampering.

Auditing: Many DFS systems keep logs of who accessed or modified files and when. This provides an audit trail for security and compliance purposes.

8. Load Balancing

To ensure optimal performance, DFS systems distribute file requests across multiple servers in a process known as load balancing. Load balancing ensures no single server is overwhelmed with too many requests, and it improves the efficiency and responsiveness of the system.

9. Heterogeneity

A DFS often operates in environments where clients and servers run different hardware platforms and operating systems. Heterogeneity support ensures:

Platform Independence: The DFS should be able to handle different types of devices and operating systems (e.g., Windows, Linux, macOS) seamlessly. This includes support for different file formats and network protocols.

Interoperability: Files should be accessible in the same format across all platforms, and applications should not need to know the underlying platform differences.

10. Naming

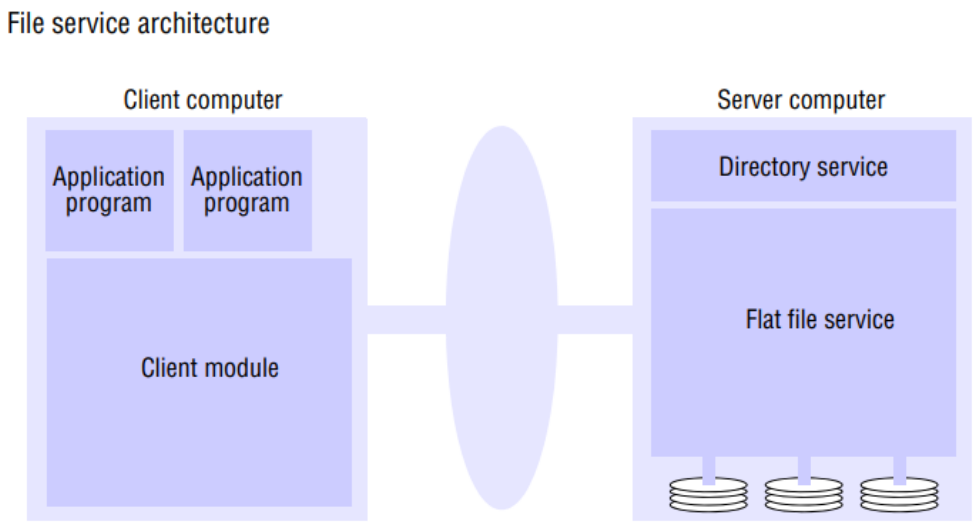
Naming is a crucial aspect of DFS because it allows users and applications to locate files easily in a distributed environment:

Global Namespace: DFS typically offers a global namespace, meaning that all files in the system can be accessed using a single, consistent path, regardless of their physical location.

c. Explain the Distributed File Service architecture.

File service architecture

In Distributed Systems (DS), the File Service Architecture refers to the design and implementation of services that provide users with access to files distributed across multiple systems. This architecture aims to ensure file sharing, consistency, transparency, and efficient access while abstracting the complexity of the underlying distribution of files across different machines or networks.



The image depicts the file service architecture, which shows the interaction between a client computer and a server computer in a distributed system.

1. Client Computer:

The client computer typically refers to the machine that requests services from a server. It has two main components:

Application Program(s): These are programs running on the client-side that require access to files or directories stored on the server. Examples of application programs could be text editors, word processors, or any software that deals with data stored remotely.

Client Module: This acts as the intermediary between the application programs and the server. It sends requests for file access to the server and processes responses. The client module handles operations like reading or writing files, retrieving directory listings, and ensuring the data is correctly requested and delivered between the client and server.

2. Server Computer:

The server provides file storage and management services to client computers. Its key components include:

Directory Service: This service manages metadata related to files. It helps locate files in the system and keeps track of information like file names, paths, and attributes. When a client requests access to a file, the directory service helps identify where that file is stored and what access permissions the client has.

Flat File Service: The flat file service is responsible for handling the actual storage and retrieval of files. It operates on the physical disks (represented by the disk stacks at the bottom). This service deals with reading, writing, deleting, and modifying the contents of files.

Communication between Client and Server:

Client-Server Interaction: The client computer interacts with the server via a network. The client module sends requests from the application programs to the server for accessing files or directories. The server's directory service first processes the request by checking the metadata (such as file location and access permissions), and the flat file service then retrieves or updates the actual file data from the storage.

Example of a Typical Workflow:

- **The client module on the client computer sends a request to access a file stored on the server.**
- **The directory service on the server checks where the file is located and if the client has permission to access it.**
- **The flat file service retrieves the requested file from storage.**
- **The file is sent back to the client module, which then makes it available to the application program that requested it.**

This architecture emphasizes the division of responsibilities between the client and server to ensure efficient management and retrieval of files in a distributed system. The server's role is to store, organize, and manage file access while the client focuses on requesting and using those files.

4 a. Explain the followings w.r.t Name Services: (a) Uniform Resource Identifiers (URIs) and (b) Uniform Resource Locators (URL).

(a) Uniform Resource Identifiers (URIs)

1. **Definition:**
   A Uniform Resource Identifier (URI) is a string of characters that uniquely identifies a resource on the internet or within a system. It is a broader concept that encompasses both URLs and URNs.
2. **Structure:**
   - **A URI typically consists of:**
     scheme:[//authority]path[?query][#fragment]
   - **Example:**
     - **http://example.com/resource (URL)**
     - **urn:isbn:0451450523 (URN)**
3. **Characteristics:**
   - **Scheme: Identifies the protocol (e.g., HTTP, FTP, mailto).**
   - **Path: Specifies the location of the resource.**
   - **Query and Fragment: Provide additional information or access specific parts of the resource.**
4. **Purpose:**
   - **URIs are used to identify resources universally, making them essential in distributed systems for locating files, services, or data.**

(b) Uniform Resource Locators (URLs)

1. **Definition:**
   A Uniform Resource Locator (URL) is a subset of URIs that not only identifies a resource but also provides the means to locate it (e.g., its address).
2. **Structure:**
   - **A URL includes:**
     scheme://host[:port]/path[?query][#fragment]
   - **Example:**
     - **https://www.example.com/page?id=123**
3. **Components:**
   - **Scheme: Protocol to access the resource (e.g., HTTP, HTTPS, FTP).**
   - **Host: Domain name or IP address of the server (e.g., www.example.com).**
   - **Port: Optional, specifies the port number (e.g., :80 for HTTP).**
   - **Path: Location of the resource on the server (e.g., /page).**
   - **Query and Fragment: Additional parameters or specific sections of the resource.**
4. **Purpose:**

- **URLs are specifically designed to locate resources on a network, such as web pages or files.**

b. What is navigation w.r.t Name Servers? Explain the following navigations wr.t Name Servers:

(a) iterative (b) multicast (c) non‐recursive server-controlled and (d) recursive server-controlled.

Navigation in Name Servers

Navigation refers to the process of resolving a name into a corresponding address or identifier using a Name Service. Name servers assist in resolving these names by communicating with each other or directly with clients. There are different methods for navigating this resolution process, as explained below:

(a) Iterative Navigation

1. **Definition:**
   - **The client interacts with multiple name servers step by step to resolve a name.**
2. **How It Works:**
   - **The client sends a request to a name server.**
   - **If the server does not have the answer, it provides a referral to another name server.**
   - **The client then contacts the referred server, repeating the process until the name is resolved.**
3. **Example:**
   - **A client resolving www.example.com might contact the root server, then the .com server, and finally the example.com server.**
4. **Advantages:**
   - **The client has control over the process.**
   - **Reduces the load on individual servers.**
5. **Disadvantage:**
   - **Requires more effort from the client.**

(b) Multicast Navigation

1. **Definition:**
   - **The client broadcasts a query to multiple name servers simultaneously.**
2. **How It Works:**
   - **The client sends a multicast request to all name servers in a specific group.**
   - **Any server with the required information responds to the client.**
3. **Example:**
   - **Used in local networks where multiple name servers exist, such as multicast DNS (mDNS).**
4. **Advantages:**
   - **Quick resolution if multiple servers are available.**
   - **Reduces dependency on a single server.**
5. **Disadvantage:**

- **Inefficient in large-scale networks due to high communication overhead.**

(c) Non-Recursive Server-Controlled Navigation

1. **Definition:**
   - **The client sends a request to the first server, and the server provides referrals for the next steps without resolving the name completely.**
2. **How It Works:**
   - **The name server returns a list of other servers that the client should query.**
   - **The client follows these referrals until the name is resolved.**
3. **Example:**
   - **A DNS server responding with "try the .com server for this query."**
4. **Advantages:**
   - **The server workload is reduced since it doesn't resolve the entire query.**
   - **Provides flexibility to the client.**
5. **Disadvantage:**
   - **Similar to iterative navigation, more effort is required from the client.**

(d) Recursive Server-Controlled Navigation

1. **Definition:**
   - **The client sends a query to one server, and that server takes full responsibility for resolving the name.**
2. **How It Works:**
   - **The name server contacts other servers on behalf of the client until it resolves the name or determines it cannot be resolved.**
   - **The server then returns the final result to the client.**
3. **Example:**
   - **A DNS resolver resolves www.example.com by contacting the root, .com, and example.com servers before replying to the client.**
4. **Advantages:**
   - **Reduces client complexity and effort.**
   - **Faster from the client's perspective.**
5. **Disadvantage:**
   - **Increases the workload on the name server.**

c. What is Domain Name System? Explain the Domain Name System with suitable example.

1. **Definition:**
    - **The Domain Name System (DNS) is a hierarchical and distributed system that translates human-readable domain names (e.g., www.example.com) into machine-readable IP addresses (e.g., 192.0.2.1) and vice versa.**
2. **Purpose:**
    - **To make it easier for users to access internet resources by using domain names instead of remembering complex numerical IP addresses.**
3. **Key Features:**
    - **Distributed: DNS is spread across multiple servers worldwide.**
    - **Hierarchical: Follows a tree-like structure with root, top-level domains, and subdomains.**
    - **Scalable: Handles billions of queries daily.**

Components of DNS

1. **Domain Names:**
    - **Hierarchical names that identify resources.**
    - **Example: www.example.com**
        - **com: Top-Level Domain (TLD).**
        - **example: Second-Level Domain.**
        - **www: Subdomain.**
2. **Name Servers:**
    - **Specialized servers that store DNS records and handle name resolution.**
    - **Types:**
        - **Root Name Servers: Handle top-level domain queries.**
        - **TLD Servers: Handle specific top-level domains (e.g., .com, .org).**
        - **Authoritative Servers: Store records for specific domains.**
        - **Caching Resolvers: Cache query results to improve speed.**
3. **DNS Records:**
    - **Store information about domain names. Common types:**
        - **A Record: Maps a domain name to an IPv4 address.**
        - **AAAA Record: Maps a domain name to an IPv6 address.**
        - **CNAME Record: Aliases one domain name to another.**
        - **MX Record: Specifies mail servers for email.**

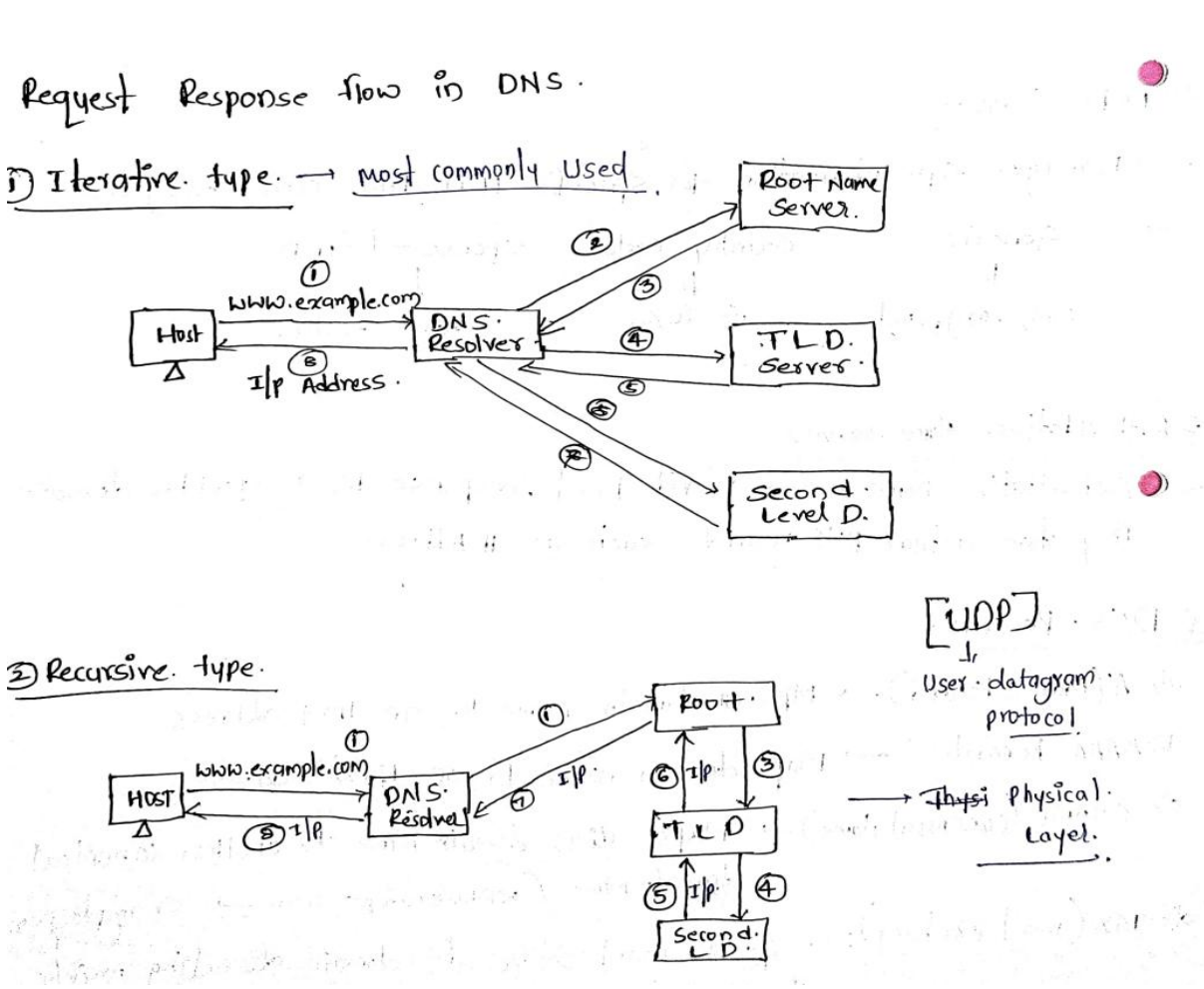
How DNS Works (Example)

Let’s resolve the domain www.example.com:

1. **Query to DNS Resolver:**
    - **A user types www.example.com in a browser. The request is sent to a local DNS resolver.**
2. **Root Server:**
    - **If the resolver doesn’t know the answer, it queries a Root Name Server, which provides the address of the TLD Server for .com.**
3. **TLD Server:**
    - **The resolver queries the TLD Server for .com, which provides the address of the Authoritative Server for example.com.**
4. **Authoritative Server:**
    - **The resolver queries the Authoritative Server, which returns the IP address for www.example.com.**
5. **Response to User:**
    - **The resolver sends the IP address back to the user's device, and the browser connects to the web server.**

Example

- **User types: www.example.com**
- **DNS resolves to: 192.0.2.1**
- **Browser connects to 192.0.2.1 to fetch the website content.**



1. Iterative DNS Request-Response Flow

- **Definition: In iterative resolution, the DNS resolver queries multiple DNS servers step by step until it finds the required IP address.**
- **Process:**
    1. **Step 1: The host (client) sends a query (e.g., www.example.com) to the DNS Resolver.**
    2. **Step 2: The resolver sends a query to the Root Name Server.**
    3. **Step 3: The Root Name Server responds with a referral to the appropriate Top-Level Domain (TLD) Server (e.g., .com server).**
    4. **Step 4: The resolver queries the TLD Server.**
    5. **Step 5: The TLD Server responds with a referral to the appropriate Second-Level Domain (SLD) server (e.g., example.com).**
    6. **Step 6: The resolver queries the SLD server, which provides the final IP address.**
    7. **Step 7: The resolver sends the IP address back to the host.**
- **Characteristics:**
    - **The resolver handles one server at a time.**
    - **The host only communicates with the resolver.**
    - **Commonly used due to lower complexity on the host side.**

---

2. Recursive DNS Request-Response Flow

- **Definition: In recursive resolution, the DNS resolver takes full responsibility for resolving the name and communicates with other servers on behalf of the client.**
- **Process:**
    1. **Step 1: The host (client) sends a query (e.g., www.example.com) to the DNS Resolver.**
    2. **Step 2: The resolver queries the Root Name Server.**
    3. **Step 3: The Root Name Server responds with the TLD Server's address.**
    4. **Step 4: The resolver queries the TLD Server.**
    5. **Step 5: The TLD Server responds with the SLD Server's address.**
    6. **Step 6: The resolver queries the SLD Server.**
    7. **Step 7: The SLD Server provides the final IP address.**
    8. **Step 8: The resolver sends the resolved IP address back to the host.**
- **Characteristics:**
    - **The resolver handles the entire process.**
    - **Simplifies the client's task but increases the resolver's workload.**

5 a. Discuss the followings: (a) Clock Skew, (b) Clock Drift and (c) Coordinated Universal Time.

Clock Skew: The difference between the times displayed by two clocks at any given moment.

Clock Drift: The gradual divergence of a clock from the correct time due to hardware imperfections.

UTC: A standard time scale used as a reference for synchronizing clocks in distributed systems. It is maintained using atomic clocks and astronomical observations

b. Explain the Cristian's method for synchronizing clocks.

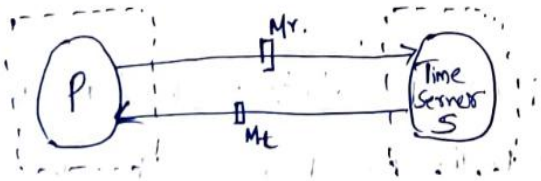
Synchronization in Synchronous System

① Cristian's method for synchronizing clocks.

→ In 1989, Cristian suggested the use of a time server connected to a device that receives signals from a source of UTC, to synchronize computers externally

→ Cristians algorithm is a fundamental method for achieving clock synchronization in DS, where multiple nodes must maintain co-ordinated time.

→ This Algorithm relies on communication with time server to adjust the clocks of client nodes.



Working of Christian's Method.

i) Initialization : Request the Time

→ A process P, needing to synchronize it's clock, sends a time request message (Mr) to time server S.

→ This message simply informs the time server that the process wants current Time.

NOTE The process records the time T_i (local time) when the request is sent.

2. Time Server Response

→ Upon receiving the request message (Mr) time server S immediately note its own current time (T_s).

→ The time server then sends a response message (mt) back to process P. containing the timestamp T_s.

3. calculated Round-Trip time (RTT).

→ When process P, receives the time servers response, it ~~notes the local servers response~~, it notes the Local time T_4 (time at which response has received).

→ Process P calculates the round-Trip Time as. (RTT → Time taken for request to travell from S to D and D to S)

$$RTT = T_4 - T_1$$

4) Estimate One way Transmission

→ Assuming delays (i.e. the time for the request msg to reach S is equal to the time for response message to return to P).

$$Delay = \frac{RTT}{2}$$

5) Synchronize the clock.

→ To synchronize its clock, process P adjusts its local time by adding the estimated one-way delay to the time T_s, received from the server.

Adjusted Time = T_s + Delay

Advantages

① simplicity
② Effective for centralized Systems.
③ Improves Synchronization Accuracy.

Limitation.

① Dependent on Delay.
② Single point failure
③ not scalable

Example

Lets Assume

- T_1 = 10:00:00.000 (Time when P sends request).
- T_4 = 10:00:00.500 (time when P receives the response).
- T_s = 12:00:00.000 (Time at server S).

① Calculate RTT

Delay = $T_4 - T_1$ = 0.500 sec.

① Calculate one way delay

Delay = RTT/2 = 0.250 sec.

Adjust Local clock.

Adjusted time = T_s + Delay
= 12:00:00:000 + 0.250
= 12:00:00.250.

The process's clock is set to.

c. What is a Logical Clock? Explain the Lamport's logical clock.

Logical Clock: A mechanism to order events in distributed systems when there is no global clock.

① Lamport Time stamp.

→ Used to order event in distributed system.

→ ensure consistency in ordering of events across d. s.

Algorithm

i) initialization

→ Each process P_i initializes its clock C_i to 0.

ii) Increment before event:

→ Before any event the process increment its counter by 1:

$C_i = C_i + 1$.

iii) Message sending.

→ when P_i sends a message to P_j it includes its current timestamp $T_s = C_i$ in the message.

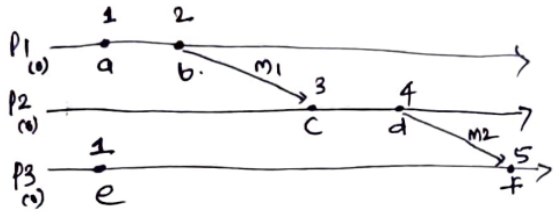
iv) Message Receiving

→ When P_j receives a msg with time stamp T_s.

⇒ P_j updates its clock. as.

$C_j = \max(C_j, T_s) + 1$

Lamport timestamps for the events



→ The figure illustrates Lamport Timestamps in a dis. with 3 process (P1, P2, P3) and events a, b, c, d, e, f.

for process P1.

1> Event a :

→ P1 starts with its clock initialized to. 1 ($C_1 = 1$

→ Time stamp for a : 1

2> Event b :

→ Before event b, P1 ~~in cle~~ increment its clock $C_1 = 2$

→ Time stamp for b : 2

→ P1 sends message ~~to~~ m1 to P2, including the time stamp 2

for process P2

1> Event c

→ P2 receives message m1 with time stamp 2

→ It updates its clock as.

$C_2 = \max(C_2, 2) + 1 = 3.$

→ P2 sends msg m2 to P3 with, stamp 4.

Process P3 :

event e.

→ P3 starts with its clock initialized to 1.

TS. e=1

event f.

→ P3 receives. msg m2 with time stamp 4

→ Update its clock. as.

$C_3 = \max(C_3, 4) + 1 = 5.$

6 a. Discuss the followings w.r.t Network Time Protocol: (a) Design aims and features (b) Modes of NTP server synchronization

NTP Overview:

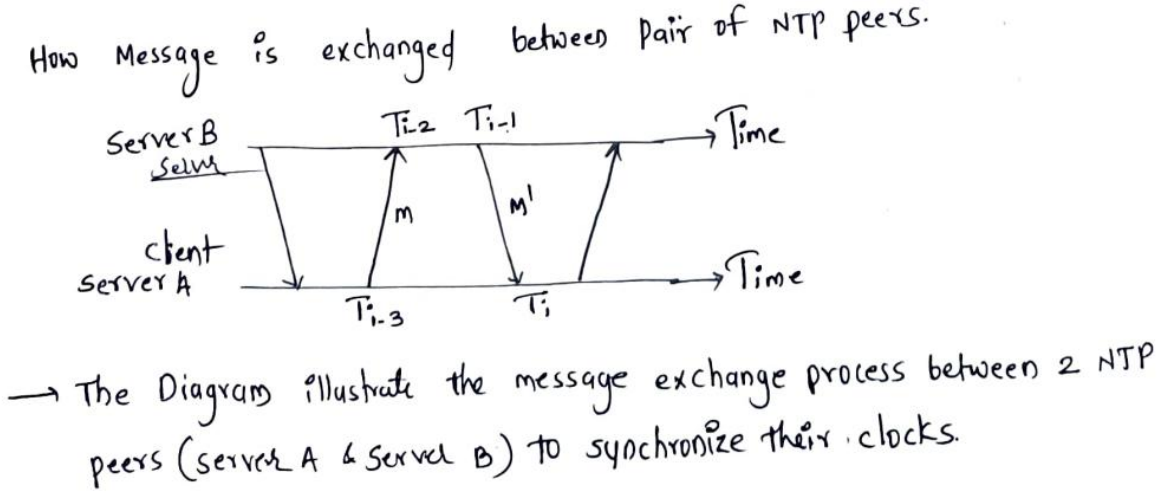
NTP is essential for synchronizing the clocks of computers over a network in a distributed system. Accurate time synchronization is crucial for coordinating activities across different systems, ensuring that events are logged in a consistent order, and managing time-dependent tasks such as database transactions and file synchronization.

NTP Hierarchical Structure:

NTP operates in a hierarchical, stratified manner, organized into different levels or "stratum."

1. **Stratum 0 (Reference Clocks)**: These are high-precision clocks, such as atomic clocks or GPS clocks, directly connected to the NTP servers. These clocks provide the base time for synchronization.
2. **Stratum 1 (Primary Time Servers)**: These servers are directly connected to Stratum 0 reference clocks. They serve as the primary time servers that distribute time to other systems.
3. **Stratum 2 (Secondary Time Servers)**: These servers synchronize their clocks with Stratum 1 servers and pass this time along to lower stratum servers and clients.
4. **Stratum 3 and lower**: These include servers and clients that synchronize time with higher stratum servers. The accuracy of time decreases as you move further down the strata, but the system remains effective for large-scale distributed environments.

NTP Message Exchange Process:



Explaination of Diagram

→ server A & server B are NTP peers exchanging messages to synchronize their clock

→ T_{i-3} : The time when Servel A sends message (m) to server B

T_{i-2} : The time when server B receives message (m) from servel A

T_{i-1} : The time when server B sends response msg (m') back to Servel A.

T_i : The time when Server A receive the response m' from servel B.

RTT

① Round Trip Delay (d):

$$d = (T_i - T_{i-3}) - (T_{i-1} - T_{i-2}).$$

Adjust:
current time + θ.

② offset

$$\theta = \frac{(T_{i-2} - T_{i-3}) + (T_{i-1} - T_i)}{2}.$$

ex:→ T_{i-3} = 10,000 sec. (when servel A sends the req).

T_{i-2} = 10.250 sec (when Server B receive the req)

T_{i-1} = 10400 sec (when server B send msg)

T_i — 10.700 sec. (when selver A rec the response)

① calculation of R.T Delay (d)

$$d = (10.700 - 10.000) - (10.400 - 10.250).$$

$$d = 0.550 \text{ sec.}$$

② calculate clock offset.

$$\theta = (10.250 - 10.000) + (10.400 - 10.700) / 2$$

$$\theta = -0.025.$$

Adjusted time = current time of selver A + θ.

= 10.700 sec. + (-0.025) = 10.675 sec. (A syn with B)

b. Explain the Global states and consistent cuts with suitable example.

In distributed systems, **global state** and **consistent cuts** are key concepts used to reason about the state of the system across multiple processes, especially when working with events or checkpoints. These concepts are essential for understanding how different components of a distributed system behave and interact over time.

1. Global State:

The global state of a distributed system refers to the state of all processes and communication channels in the system at a particular point in time. Since there is no global clock in a distributed system, the global state cannot be directly observed or captured at a single moment. Instead, it must be inferred by examining the states of individual processes and their communication messages.

Components of Global State:

- **Local states of processes**: The local state of each process includes its variables and the state of its execution (e.g., instruction being executed, values of variables).
- **Messages in transit**: Since messages are being passed between processes, the global state must also account for messages that are in transit between processes but have not yet been received.

2. Consistent Cuts:

A **cut** in a distributed system is a subset of events in the system that represents the state of the system at a particular point in time. It is essentially a snapshot of the system's state across all processes and communication channels. A cut is **consistent** if it reflects a valid execution of the system, meaning the events in the cut obey the causality constraints of the system.

Key properties of a consistent cut:

- A consistent cut must respect the **happens-before** relationship, which is a causal ordering of events.
- If an event in one process causally depends on an event in another process, the event from the first process must appear before the event from the second process in any consistent cut.

Example of Global States and Consistent Cuts:

Consider a distributed system with two processes (P1 and P2) and a communication channel between them. Let's say that:

1. **P1** performs an action (e.g., sends a message m).
2. **P2** receives the message m and performs an action (e.g., processes the message).

Global State at Different Points in Time:

- **At Time 1**: P1 is at state s1, and P2 is at state s2. No message has been sent or received.

Global state = (P1 in state s1, P2 in state s2, no messages in transit)

- **At Time 2**: P1 sends message m to P2, but P2 has not yet received the message.

Global state = (P1 in state s2, P2 in state s2, message m in transit)

- **At Time 3**: P2 receives the message m and processes it.

Global state = (P1 in state s3, P2 in state s4, message m delivered and processed)

Cuts in the Distributed System:

- **Cut 1 (consistent)**: We could take a cut after P1 sends the message but before P2 receives it. In this case, the global state of the system would capture the state of P1 and P2, with the message in transit. This cut is consistent because it reflects a valid state where P1 has sent the message, but P2 has not yet received it.
- **Cut 2 (inconsistent)**: A cut taken where P2 is in the state after receiving the message, but P1 is still in the state before sending the message, would be **inconsistent**. This is because it violates the causality constraint — P2 cannot process the message before it is sent by P1.

Important Points:

- A **consistent cut** reflects a possible history of the distributed system where the events are causally consistent with one another.
- An **inconsistent cut** would represent a situation that cannot possibly occur due to the inherent causality constraints of the system.

Real-world Example:

Consider a distributed file system where Process A is writing to a file and Process B is reading the file. Let's say:

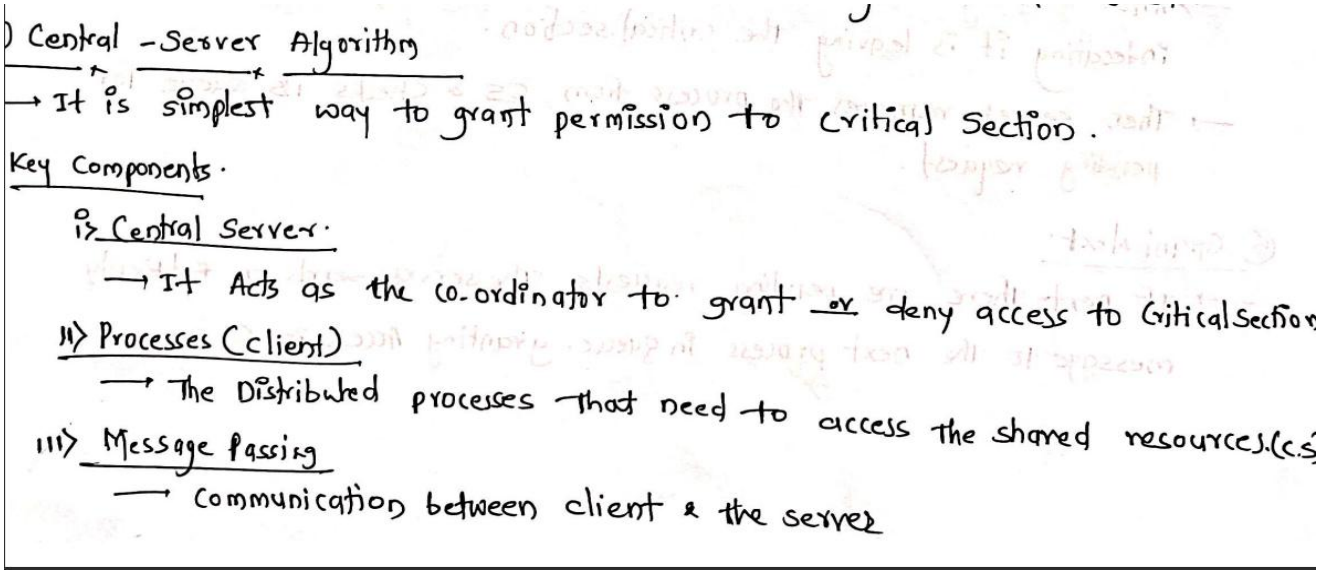
- Process A writes the data at time t1.
- Process B reads the data at time t2, after Process A has written it.

If you take a global state at time t1 (after A writes the data but before B reads it), that would be a consistent cut, because Process A's write event causally precedes Process B's read event. However, if you took a cut where Process B reads the data before Process A writes it, that would be an inconsistent cut, as it would violate the causal ordering (B cannot read data before A writes it).

Importance of Consistent Cuts:

- **Checkpointing and recovery**: Consistent cuts are used in distributed systems to create checkpoints that can be used for recovery in case of failure. A consistent cut ensures that the system can be restored to a valid state after recovery.
- **Deadlock detection**: Consistent cuts help in detecting deadlocks and race conditions by providing a way to examine the state of the system across processes.
- **Logging**: In systems that require distributed logging, consistent cuts are used to ensure that the logs reflect the correct ordering of events.

6. a. Discuss the followings algorithms for mutual exclusion in Distributed Systems: (a) central server algorithm (b) ring-based algorithm (c) multicast and logical clocks.



) Central - Server Algorithm

→ It is simplest way to grant permission to Critical Section.

Key Components.

i> Central Server.

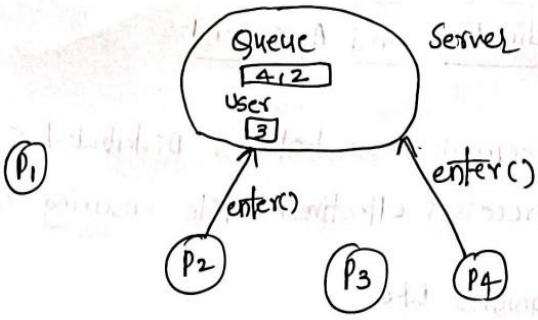
→ It Acts as the co-ordinator to grant or deny access to CriticalSection

ii> Processes (client)

→ The Distributed processes that need to access the shared resources.(c.s

iii> Message Passing

→ communication between client & the server



steps

① Request

→ when process wants to enter critical section, it sends a Request msg to the serv. This request include process ids.

→ Request will be sent using enter() function

② Grant Access

→ when process wants to enter critical section, ~~it sends a Req~~ Server sends a Reply message to requesting process, grating access

— If Another process is already in critical section! The server queue the request.

③ Critical section Execution

→ The process enters the critical section and performs its operation.

④ Release

→ After finishing that task process sends release message to server indecating it is leaving the critical section.

— Then server removes the process from CS & checks its queue for pending request.

⑤ Grant Next.

→ If next there are pending requests, The server sends a ~~Rpl~~ Reply message to the next process. in queue. granting Access in C.S.

② Ring based Algorithm in Mutual Exclusion

steps

① Logical ring structure

→ A group of processes it arranged in a logical ring, meaning that each process P_j is connected to exactly one other process in a circular manner (clockwise).

② Token Passing

→ A Unique token (ex:- small message) circulates through the ring

→ This token is requeired for a process to access the CS.

→ If a process do not need to enter cs. it immediatly forwards the token to its next neighbour.

③ Entering the Critical section

→ If a process requires access to the C.S. it waits until it receives the token.

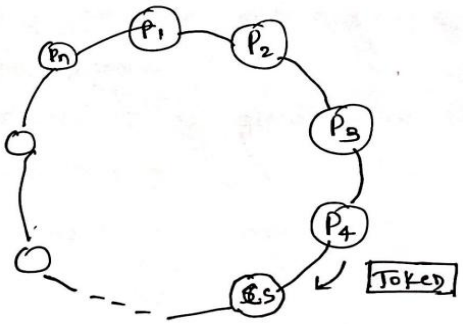
→ Once the token is received the process retains it and enters the C.S.

④ Exiting the Critical section

→ After completing its operation in C.S. the process sends the token to its next neighbour.

⑤ Direction

→ The token is passed in single direction (clockwise).



The Diagram Represents RingBased Algorithm for ensuring mutual exclusion in D.S.

① The processes ($P_1 \dots P_n$) are Arranged in a logical ring..
Each process communicates only with its immediatly neighbour process in clockwise.

② Token is key element of algorithm. It grants permission to the process holding it to enter to C.S.

③ The C.S (Critical section) is a part of the process's code where shared Resources are accessed. only one process → CS.
Diagram shows Process (P_4) having token. indecating that P_4 is currently in C.S.

④ After P_4 completes its work the token is passed to next process in ring.

b. What are the properties of Reliable multicast? Explain the Reliable multicast algorithm.

Properties of Reliable Multicast:

Reliable multicast refers to the communication protocol where data is sent from one sender to multiple receivers in a multicast group, and the protocol ensures that the data is reliably delivered to all the receivers, even in the presence of network failures or other issues. The following are key properties of a **reliable multicast**:

1. **Message Delivery**:
    - **Reliable delivery** ensures that all messages sent to a multicast group are successfully received by all members, regardless of network failures or congestion. If any receiver fails to receive a message, it must be retransmitted.
2. **Ordered Delivery**:
    - **Message order preservation** guarantees that messages are delivered in the same order they were sent, ensuring that the receiving processes can correctly interpret the data. This is especially important for applications like video streaming or collaborative systems.
3. **Duplicate Prevention**:
    - **Duplicate suppression** ensures that each message is delivered only once to each receiver, even in the case of retransmissions due to network failures or congestion.
4. **Fault Tolerance**:
    - **Fault tolerance** guarantees the system can handle receiver failures (e.g., receivers may join or leave the multicast group) and still maintain reliable delivery. This may involve mechanisms for handling the loss of messages or dynamically adapting to changes in the group membership.
5. **Scalability**:
    - **Scalability** ensures that the reliable multicast protocol works efficiently even when the number of receivers in the multicast group grows large, without requiring an excessively high amount of resources from the sender or the network.
6. **Congestion Control**:
    - **Congestion control** ensures that the protocol adjusts its transmission rate to avoid overwhelming the network, which can be important in systems with large groups of receivers.
7. **Receiver Acknowledgment**:
    - **Acknowledgment mechanism** involves receivers confirming that they have received messages. This can be done via either individual or group-based acknowledgments to allow the sender to retransmit lost packets.

Reliable Multicast Algorithm:

A reliable multicast algorithm ensures that messages sent from a source (sender) to multiple receivers are delivered reliably, ordered correctly, and free from duplicates. Several algorithms can be used for this purpose, including **receiver-based approaches**, **sender-based approaches**, and **hybrid approaches**. One well-known algorithm is the **Receiver-Driven Reliable Multicast (RDM) protocol**.

Key Features of the Reliable Multicast Algorithm:

1. **Multicast Group Setup**:

- The sender broadcasts a message to a multicast group (set of receivers). The message includes an identifier and other essential information for the receivers to know which group they belong to.

2. **Receiver Acknowledgment**:
   - Each receiver acknowledges the receipt of a message. There are several methods to handle acknowledgment:
     - **Receiver-based acknowledgment**: Receivers send individual acknowledgment messages for each message they receive, informing the sender.
     - **Receiver set-based acknowledgment**: Receivers wait for a specific time before acknowledging, reducing the number of acknowledgment messages sent to the sender.
     - **Feedback suppression**: This mechanism is used to prevent overload on the sender by suppressing acknowledgment messages until necessary.
3. **Message Retransmission**:
   - If a sender does not receive an acknowledgment (due to packet loss, receiver failure, etc.), it retransmits the message to all members of the multicast group.
   - Retransmissions can be triggered by a timeout (i.e., if the sender does not receive an acknowledgment within a specified time frame).
4. **Ordering of Messages**:
   - To maintain **message ordering**, a sequence number is typically added to each multicast message. The sequence number ensures that messages are delivered in the correct order, even if they are received out of order due to network delays or retransmissions.
   - Some algorithms may also implement **timestamping** to ensure proper sequencing.
5. **Fault Recovery**:
   - The algorithm should support mechanisms to handle receiver failures. If a receiver does not acknowledge a message, it is assumed to have missed it, and the sender may resend it.
   - Some protocols use **NACK (Negative Acknowledgment)** messages from receivers, indicating missing messages, so the sender can retransmit only the missing data.
6. **Scalability and Optimization**:
   - In large multicast groups, it is inefficient for every receiver to send individual acknowledgment messages. Instead, the system can use hierarchical acknowledgment structures or **feedback trees**, where a set of receivers reports back to a designated node (e.g., a root node) that then forwards a consolidated acknowledgment to the sender.
   - **Receiver clustering** or **group-based acknowledgment** reduces the number of messages exchanged, making the protocol more scalable.

Example of Reliable Multicast Algorithm:

1. **Sender Initiates Message**: The sender sends a multicast message to a group of receivers. Each message is tagged with a unique sequence number.
2. **Receiver Receives Message**: Each receiver that successfully receives the message sends back an acknowledgment (ACK) to the sender. The ACK could be sent directly to the sender or be forwarded via a feedback mechanism.

3. **Timeout or Missing Acknowledgment**: If the sender does not receive an acknowledgment from a receiver within a specified time, the message is retransmitted.
4. **Receiver Notifies Sender of Losses**: In some algorithms, receivers can explicitly notify the sender of missing messages using NACKs (Negative Acknowledgments). The sender can then retransmit only the missing messages, improving efficiency.
5. **Ordering and Duplicates**: The algorithm ensures that messages are received in the correct order by using sequence numbers. Duplicates are detected by the receivers (based on sequence numbers) and discarded to ensure only one copy of each message is processed.
6. **Receiver Joins or Leaves**: The system handles changes in the group, such as receivers joining or leaving the multicast group, by updating the state of the sender and the receivers, ensuring continued reliability

8 a. Explain the Maekawa's voting algorithm for mutual exclusion in Distributed Systems.

On initialization
state := RELEASED;
voted := FALSE;

For p_i *to enter the critical section*
state := WANTED;
Multicast *request* to all processes in V_i;
Wait until (number of replies received = K);
state := HELD;

On receipt of a request from p_i *at* p_j
if (*state* = HELD *or voted* = TRUE)
then
queue *request* from p_i without replying;
else
send *reply* to p_i;
voted := TRUE;
end if

For p_i *to exit the critical section*
state := RELEASED;
Multicast *release* to all processes in V_i;

On receipt of a release from p_i *at* p_j
if (queue of requests is non-empty)
then
remove head of queue – from p_k, say;
send *reply* to p_k;
voted := TRUE;
else
voted := FALSE;
end if

b. What is an Election algorithm? What are its requirements? Explain the ring-based election algorithm.

Elections Algorithms

→ Algorithm to choose a unique process to play a particular role is called an Election Algorithm. (Role → Coordinator → Maintain other processes & Task)

→ Ex:- Central server for mutual exclusion, one ~~server~~ process ~~must~~ will be elected as a sever everybody must agree.
if server wants to retire, then another election is required to choose a Replacement.

To elect the processes there are Two Algorithms.
i> Ring Based Election Algorithm.
ii> Bully Algorithm.

i> Ring Based Election Algorithm.

The Ring Based election Algorithm is distributed Algorithm used to select a co-ordinator in ring structure.

steps in Algorithm.

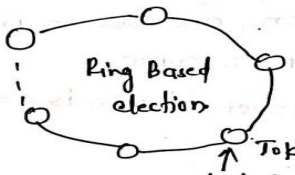
step 1: Initialization: Each node in the n/w is assigned unique IDs.

step 2: Message Passing: The Algorithm begins when node initiats an election process. It sends a special message often all as election message or token. containing its (id). to neighboring nodes in the ring.

step 3: Comparison & forwarding
→ Upon receiving the token, each node compares the identifier in the message with its own. (id)
i> if the received ID is greater than its own. it forwards the msg to next node in ring
ii> if the received ID is smaller than its own, it simply sends msg to next node.
iii> If the id in message matches with its own id, it conclude that it has the higest id. and declare itself As a co-ordinator.

---

step 4: The New co-ordinatior sends a 'co-ordinator message' arround the ring to inform all the processes of its identity.
Each process updates its record of co-ordinator.



ex:→

Imagine a ring with four processes P1, P2, P3, & P4 having ID's 1, 2, 3 & 4

→ P2 Detects that old co-ordinator has failed & initiates the election
→ P2 sends an election message with ID2 to P3.
→ P3 Updates the message with ID3 and sends it to P4
→ P4 Updates the message with ID 4 and sends to P1
→ P1 forwards the message with ID 4 back to P2
→ P2 recognizes that ID 4 is the higest and declare P4 as the co-ordinator

Advantages ① simple to implement
② suited for sys with logical ring structure.

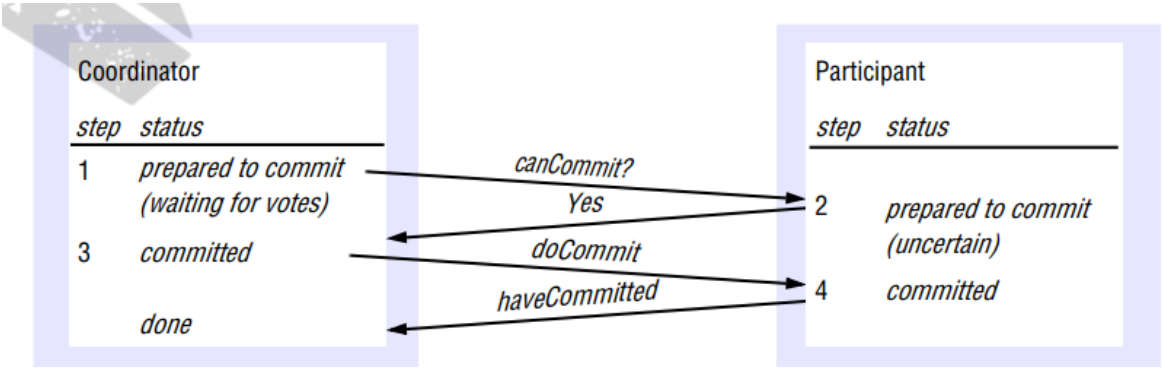
9 a. Explain the two-phase commit protocol w.r.t distributed transactions.

The **Two-Phase Commit (2PC) Protocol** is a widely used **consensus protocol** in distributed systems to ensure **atomicity** and **consistency** of transactions across multiple nodes in a distributed environment. It is primarily used to manage transactions that span multiple resources or databases, ensuring that all participants in a transaction either **commit** or **abort** the transaction in a coordinated manner.

The protocol consists of two main phases:

1. **Phase 1 - The Prepare Phase** (also known as the voting phase)
2. **Phase 2 - The Commit Phase** (also known as the decision phase)

Overview of the Two-Phase Commit Protocol



1. Coordinator and Participant Roles:

- **Coordinator:** Manages the transaction by orchestrating the commit process across all participants.
- **Participant:** A distributed system node involved in the transaction. Each participant decides whether it can commit and informs the coordinator.

Steps in the Diagram:

Step 1: Coordinator Initiates ("Prepared to Commit")

- The **coordinator** sends a canCommit? message to all participants, asking if they are ready to commit the transaction.
- At this point, the **coordinator** waits for votes (responses) from the participants.

Step 2: Participant Votes ("Prepared to Commit - Uncertain")

- The **participant** receives the canCommit? message and decides if it can commit based on its state (e.g., resource availability, data consistency).
- If it can commit, the participant responds with Yes and transitions to the **"Prepared to Commit" (uncertain)** state.
    - **Uncertain State:** The participant is ready but cannot finalize the transaction independently—it waits for the coordinator's decision.

Step 3: Coordinator Decides ("Committed")

- If all participants vote Yes, the **coordinator** sends a doCommit message to all participants, instructing them to commit the transaction.

- The **coordinator** transitions to the **"Committed"** state.

Step 4: Participant Commits ("Committed")

- Upon receiving the doCommit message, each participant commits the transaction and transitions to the **"Committed"** state.
- The participant acknowledges this by sending a haveCommitted message back to the coordinator.

Final State: Coordinator Completes ("Done")

- Once the coordinator receives haveCommitted acknowledgments from all participants, it transitions to the **"Done"** state, indicating the transaction is fully committed.

b. Discuss the various methods of concurrency control in distributed transactions.

Key Methods of Concurrency Control

1. Lock-Based Methods

Locks are used to control access to shared data.

- **Two-Phase Locking (2PL):**
    - Transactions acquire locks in a growing phase and release them in a shrinking phase.
    - Guarantees serializability but may lead to deadlocks.
- **Centralized Lock Manager:**
    - A single node manages all locks in the system.
    - Simple to implement but can become a bottleneck.
- **Distributed Lock Manager:**
    - Locking is distributed across multiple nodes.
    - Increases fault tolerance but adds coordination overhead.

---

2. Timestamp-Based Methods

Each transaction is assigned a unique timestamp to determine its execution order.

- **Basic Timestamp Ordering:**
    - Transactions are executed in the order of their timestamps.
    - Ensures serializability but may lead to frequent transaction rollbacks.
- **Multiversion Timestamp Ordering (MVTO):**
    - Maintains multiple versions of data items.
    - Readers access older committed versions, and writers create new ones.
    - Avoids blocking but increases storage overhead.

---

3. Optimistic Concurrency Control (OCC)

Transactions proceed without restrictions and validate at commit time.

- **Phases:**
    1. **Read Phase:** Transactions read data without locking.
    2. **Validation Phase:** Ensures no conflicts with other transactions.
    3. **Write Phase:** Applies updates if validation succeeds.
- **Use Case:** Suitable for low-contention environments.
- **Limitation:** High abort rates in high-contention scenarios.

---

4. Multiversion Concurrency Control (MVCC)

Maintains multiple versions of each data item.

- **How it Works:**

    - Each transaction sees a consistent snapshot of the database.
    - Writers create new versions, while readers access existing ones.
- **Advantages:**
    - Readers do not block writers, and vice versa.
- **Disadvantages:**
    - Increased storage requirements due to multiple versions.

---

5. Quorum-Based Methods

Uses voting among replicas to ensure consistency.

- **Read Quorum (R):** Minimum number of replicas required for a read operation.
- **Write Quorum (W):** Minimum number of replicas required for a write operation.
- Ensures consistency by overlapping R and W (i.e., R+W>NR + W > NR+W>N, where NNN is the total number of replicas).

---

6. Graph-Based Concurrency Control

- Transactions are represented as nodes, and dependencies between them as edges.
- The system prevents or resolves cycles in the graph to avoid deadlocks.

---

7. Hybrid Methods

Combines different approaches, such as:

- **Locking and OCC:** Use locking for short transactions and OCC for long transactions.
- **Timestamp and MVCC:** Use timestamps for conflict resolution and MVCC for improved performance.

Comparison of Methods

Method	Advantages	Disadvantages
Lock-Based	Simple and widely used	Deadlocks and blocking
Timestamp-Based	No deadlocks	High rollback rates
Optimistic Concurrency	Best for low-contention systems	High abort rates in high contention
MVCC	No blocking for readers	High storage overhead
Quorum-Based	High fault tolerance	Increased communication costs

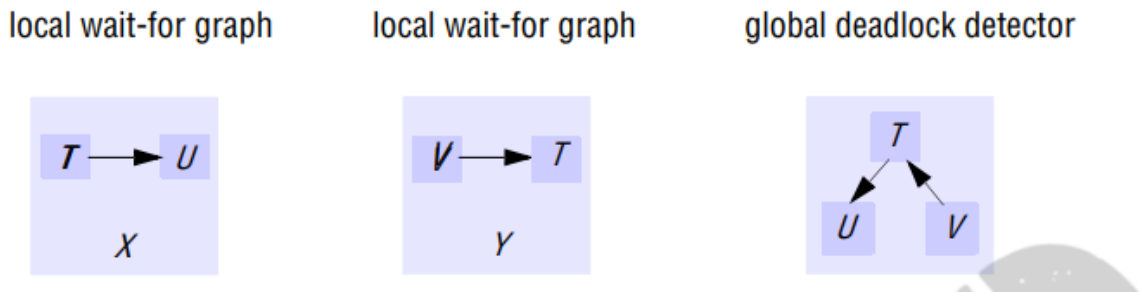
10 a. Discuss (a) Phantom deadlocks and (b) Edge chasing w.r.t deadlock in Distributed Systems

Phantom Deadlocks in Distributed Systems

A **phantom deadlock** is a situation where a deadlock is falsely detected by a deadlock detection mechanism, even though no real deadlock exists. Phantom deadlocks primarily occur in **distributed systems** due to delays in propagating wait-for information across servers.

How Phantom Deadlocks Occur

1. **Wait-for Graphs:** Distributed deadlock detection involves maintaining and analyzing **wait-for graphs** (WFG) that represent transaction dependencies across servers.
    - Nodes in the graph represent transactions.
    - Edges represent a transaction waiting for a resource held by another transaction.
2. **Delay in Updates:**
    - Information about resource release or transaction termination may take time to propagate between servers.
    - During this delay, a deadlock detection algorithm might process outdated WFG information.
3. **Breaking the Cycle:**
    - If one of the transactions in the cycle releases a lock or aborts during the detection process, the cycle no longer represents a deadlock.
    - However, the detection mechanism may still falsely identify the cycle as a deadlock.



Edge Chasing for Distributed Deadlock Detection

Edge chasing (or path pushing) is a distributed technique for detecting deadlocks in a system where resources and transactions span multiple servers. Instead of constructing a global **Wait-For Graph (WFG)**, edge chasing relies on forwarding "probe messages" along potential dependency paths to detect cycles in the distributed wait-for relationships.

Key Concepts in Edge Chasing

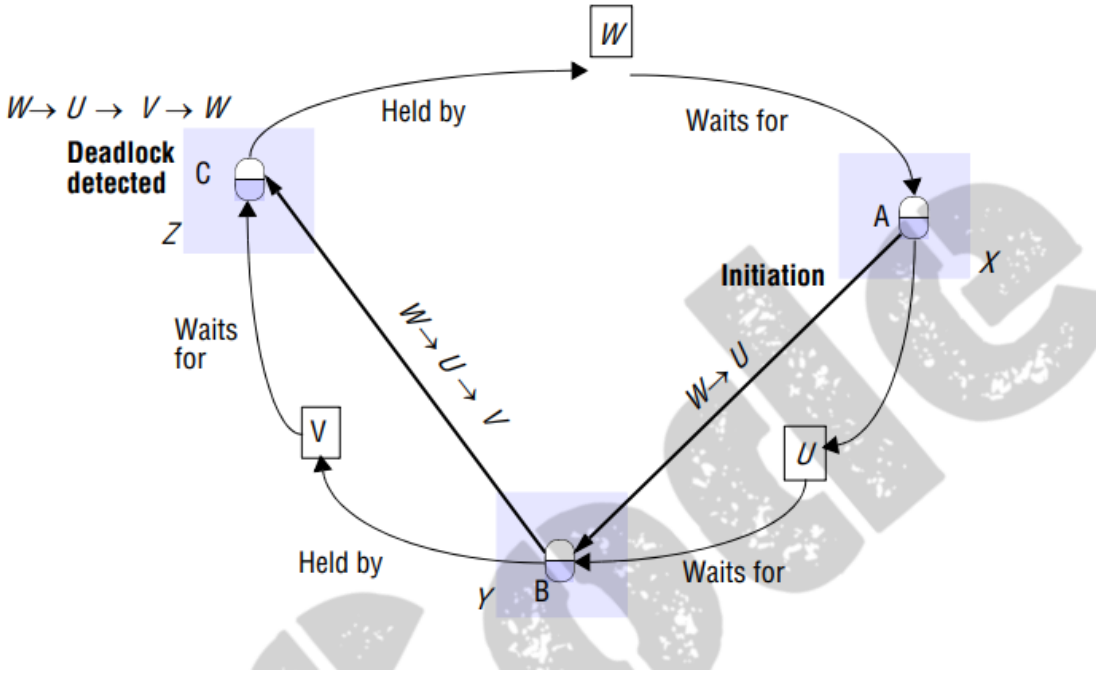
1. **Local Wait-For Graphs:**
    - Each server maintains its own **local wait-for graph** that records transaction dependencies (edges) within its domain.
2. **Probe Messages:**
    - Probes are special messages sent between servers to trace transaction dependencies across the distributed system.

- A probe carries a path (sequence of wait-for relationships) that represents a potential cycle in the global wait-for graph.

3. **Cycle Detection:**
    - A deadlock is detected when a probe message returns to the server that initiated it, forming a cycle in the global wait-for graph.

Steps in Edge Chasing

1. **When to Send a Probe:**
    - A server sends a probe whenever it adds a new edge T1→T2T1 \to T2T1→T2 to its local wait-for graph, and T2T2T2 is waiting for a resource held by another transaction T3T3T3 at a remote server.
2. **Probe Format:**
    - The probe includes:
        - The initiating transaction.
        - The current dependency path (e.g., T1→T2→T3T1 \to T2 \to T3T1→T2→T3).
3. **Forwarding Probes:**
    - When a probe reaches a server, the server checks whether:
        - The probe forms a cycle by returning to its initiator.
        - The transaction at the server is waiting for another resource.
    - If the transaction is waiting, the server extends the path in the probe and forwards it to the next server in the dependency chain.
4. **Cycle Detection:**
    - A cycle is detected when the initiating transaction appears in the probe's path.
5. **Deadlock Resolution:**
    - When a deadlock is detected, one or more transactions in the cycle are aborted to break the deadlock.


W
W→ U → V → W
Held by
Waits for
Deadlock detected
C
Z
A
X
Initiation
Waits for
W→ U→ V
W→ U
V
U
Held by
Waits for
Y
B

b. Explain the following approaches used in the file recovery in Distributed Systems: (a) Logging and (b) Shadow versions.

(a) Logging

Logging is a technique where changes to files are recorded in a log file before being applied to the actual file. This log acts as a recovery mechanism in case of failure.

How Logging Works:

1. **Write-Ahead Log (WAL):**
    - Changes (write or update operations) are first recorded in a log before being applied to the actual file.
    - The log entries are persistent and are stored in stable storage.
2. **Types of Logs:**
    - **Redo Logs:** Store the changes required to redo operations in case of a crash.
    - **Undo Logs:** Store the original state of the data to undo operations if needed.
3. **Recovery Process:**
    - After a failure, the system uses the logs to either:
        - **Redo operations** that were committed but not applied to the file.
        - **Undo operations** that were applied to the file but not committed.

Advantages:

- Provides durability and atomicity.
- Ensures consistency even if a failure occurs during file operations.

Disadvantages:

- Additional storage overhead for logs.
- Slower performance due to logging operations before actual file changes.

(b) Shadow Versions

Shadow Versions involve creating a copy (or "shadow") of a file before making any changes. If a failure occurs during the operation, the original version remains intact.

How Shadow Versions Work:

1. **Version Creation:**
    - Before modifying a file, a shadow (backup) copy of the file is created.
    - All updates are performed on the shadow copy.
2. **Commit or Rollback:**
    - If the operation completes successfully, the shadow copy replaces the original file.
    - If a failure occurs, the shadow copy is discarded, and the original file remains intact.

Advantages:

- Simple and effective for ensuring consistency.
- No need to track changes or maintain logs.

Disadvantages:

- Significant storage overhead as a complete copy of the file is required.
- Inefficient for large files or frequent updates.